

Deep Limit Order Book Forecasting

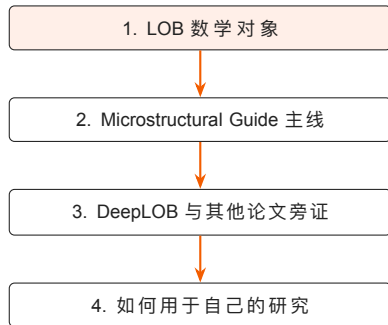
以微观结构为主线的入门讲解

Yida Ding May 26, 2026

■ 40 分钟讲清楚什么

听完要带走的三个问题

- ▶ LOB 预测到底在预测什么？
- ▶ 为什么 tick size、spread、horizon 会改变预测难度？
- ▶ 为什么分类准确率高，不等于交易信号可用？



■ 五篇论文怎么放在一起

论文	核心作用	本次讲法
Microstructural Guide	主论文：用市场微观结构解释 LOB 预测什么时候有效	重点讲数学对象、标签、tick regime、MCC、 p_T
DeepLOB	经典模型：把 LOB 当作有局部结构的时间序列	讲模型思想，不展开工程细节
Ait-Sahalia et al.	证明高频可预测性存在，但很快衰减	一页说明信号寿命
HLOB	用图结构表达 LOB 内部依赖	作为结构先验例子
Benchmark Study	说明单一数据集高分不可靠	作为泛化风险提醒

LOB 是什么：一个状态快照

- ▶ LOB 记录还没有成交的买单和卖单。
- ▶ 每一档有价格和数量：卖一、卖二、买一、买二等。
- ▶ 深度学习常用前 $L = 10$ 档，把一个快照写成向量。

一条 LOB 记录

$$L(\tau) = \{p_\ell^{ask}, v_\ell^{ask}, p_\ell^{bid}, v_\ell^{bid}\}_{\ell=1}^L \in \mathbb{R}^{4L}.$$

模型看到的输入

$$X_\tau = (L(\tau - T + 1), \dots, L(\tau))$$

$$X_\tau \in \mathbb{R}^{T \times 4L}$$

直觉

不是一根价格线，而是一段“买卖盘形状”的电影。

■ 预测目标：未来中间价方向

中间价与价差

$$m_\tau = \frac{p_1^{ask}(\tau) + p_1^{bid}(\tau)}{2}$$
$$\sigma_\tau = p_1^{ask}(\tau) - p_1^{bid}(\tau)$$

- ▶ m_τ : 常用来定义方向。
- ▶ σ_τ : 买卖价差，也是交易摩擦。

三分类标签

$$y_\tau = \begin{cases} -1, & m_{\tau+\Delta\tau} - m_\tau \leq -\theta, \\ 0, & |m_{\tau+\Delta\tau} - m_\tau| < \theta, \\ +1, & m_{\tau+\Delta\tau} - m_\tau \geq +\theta. \end{cases}$$

- ▶ θ : tick size, 最小价格变动单位。
- ▶ $\Delta\tau$: 未来多少次 LOB 更新，而不是固定秒数。

■ 主论文到底研究什么

实验设置

- ▶ 15 只 NASDAQ 股票，2017–2019 年。
- ▶ LOBSTER 逐笔订单簿数据。
- ▶ 输入：最近 100 次 LOB 更新，前 10 档。
- ▶ 输出： H_{10}, H_{50}, H_{100} 后的方向。

真正的问题

- ▶ 不只是“DeepLOB 分数多少”。
- ▶ 而是：哪些股票更容易预测？为什么？
- ▶ 分类指标好，是否意味着信号可以形成完整交易？



LOBFrame: 把流程标准化

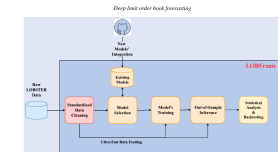


Figure 2. Pipeline representation of LOBFrame. This framework facilitates the Limit Order Book (LOB) forecasting practice through a workflow pipeline that includes data cleaning, standardized preprocessing, model selection, and integration of both existing and novel forecasting models, and ultra-fast data feeding. It enables comprehensive model training, out-of-sample inference, statistical analysis, and robust backtesting, offering the academic and practitioner communities a benchmark tool for advancing research and standardizing practices in the field. The use of distinct colors in the diagram highlights logically separated stages of the pipeline.

assumption. Indeed, weights are learned during inference, and derived features (i.e., learned from the training set) are data-adaptive. The LSTM layer, on the other side, is used to capture residual time dependencies among the resulting features. It is worth underlining that short time dependencies are already captured by the convolutional layers, which take LOB samples as inputs (see Figure 1). To train, validate and test the DeepLOB model, we design a high-performance data loader, which samples mini-batches of size 32 (as per the original model's implementation), each made of inputs with size 300×40 . Dimension 300 (i.e., the temporal dimension) represents the history length and corresponds to the number of historically consecutive LOB updates constituting each sample. Dimension 40, instead, is the number of spatial constraints for each LOB's samples (see equation (1)). The sampling process differs for the training, validation, and test sets. During training, the (sub)-sampling is random and balanced. From each trading day, we draw the number of samples for the less represented class and (i) if this value is ≥ 5000 , then we sample 5000 random representatives (a representative is a 100×40 input) for each of the three classes (see equation (21)), otherwise, (ii) if this value is < 5000 , then we sample a number of random representatives for each class which is equal to the number of samples for the less represented class.

During validation and test stages, we still sample batches with a size of 32, but they are always sequential and cover the totality of data in the two sets. In line with the related literature (Zhang et al., 2019), the model is always trained for a maximum number of epochs equal to 100, with patience equal to 15 epochs. We use a modified version of the Adam optimizer (Kingma and Ba 2014) with decoupled weight decay (Loshchilov and Hutter 2017), commonly known as "AdamW". Following the latest applied research findings

(Brown et al., 2020; Karpathy, 2020), we use a learning rate equal to 6×10^{-4} , a β_1 decay rate equal to 0.9 and a β_2 decay rate equal to 0.99. The choices of values for these parameters are determined by the training pipeline described above, which is different from the one proposed in the original work (Zhang et al., 2019) and relies on a smaller number of training samples to reduce the model's exposure to the noise characterizing the LOB. The entire framework described in this paper is coded in Python using the PyTorch deep learning library (Paszke et al., 2018). A total number of 135 experiments have been run on the University College London Computer Science Department's High-Performance Computing Cluster (UCL, 2024) for a cumulative GPU runtime of 959 hours, 16 minutes and 21 seconds. Six different types of GPUs have been used: (i) NVIDIA GeForce GTX 1080 Ti; (ii) NVIDIA GeForce RTX 3080 Ti; (iii) NVIDIA TITAN X (Pascal); (iv) NVIDIA TITAN Xp; (v) Tesla V100 PCIe-16GB; and (vi) Tesla V100 PCIe-32GB.

6. Microstructural prices

In this section, we investigate stocks' foundational microstructural properties, focusing on the interplay between tick size and bid-ask spread, liquidity at the best levels, and the structural organization of the Limit Order Book (LOB). Through this analysis:

- We identify a practical classification standard for tick-based stock categories, identifying unique behavioral patterns across small-, medium-, and large-tick stocks and revealing the macro-stability of spread distributions over the 3-year analysis period.

Source: Briola et al., "Deep Limit Order Book Forecasting: A Microstructural Guide", Figure 2.

■ Tick size: 这篇文章的核心变量

按价差相对 tick 分类

$$\bar{\sigma} = \mathbb{E}[\sigma_{\tau}], \quad r = \frac{\bar{\sigma}}{\theta}$$

small-tick	$\bar{\sigma} > 3\theta,$
medium-tick	$1.5\theta < \bar{\sigma} < 3\theta,$
large-tick	$\bar{\sigma} < 1.5\theta.$

- ▶ large-tick: 价差常接近 1 tick, 最优档队列很重要。
- ▶ small-tick: 价差可以展开成很多 tick, 结构更稀疏、更不稳定。
- ▶ 因此, 同样的标签规则, 在不同股票上不是同一个难度。

初学者理解

tick size 不只是交易所规则; 它决定“价格动一下”在这个市场里有多大意义。



signal processing [54]. FIR filters are popular smoothing techniques for denoising target signals and they are simple to implement and work with. We can write any FIR filter in the following form:

$$y(n) = \sum_{k=0}^M b_k x(n-k) \quad (5)$$

where the output signal $y(x)$ at any time is a weighted sum of a finite number of past values of the input signal $x(x)$. The filter order is denoted as M and b_k is the filter coefficient. In a convolutional neural network, the coefficients of the filter kernel are not obtained via a statistical objective from traditional signal filtration theory, but are left as degrees of freedom which the network infers so as to estimate its value function at control.

The details of the first convolutional layer inevitably need some consideration. As convolutional layers operate a small kernel to "scan" through input data, the layout of input order book information is vital. Recall that we take the most 100 recent updates of an order book to form a single input and there are 40 features per time stamp, so the size of a single input is $1000 \times 40 = 40,000$ (remember that 20 features are 0-1000000).

$$(g^{(1)}(t), r^{(1)}(t), g^{(0)}(t), r^{(0)}(t))_{t=0}^{t=2\pi} \quad (9)$$

where i denotes the i -th level of a limit-order book. The size of our first convolutional filter is $[1 \times 2]$ with stride $[1 \times 2]$. The first layer essentially summarizes information between price and volume $\{p^{(i)}, v^{(i)}\}$ at each order book level. The usage of stride is necessary here as an important property of convolutional layers is parameter sharing. This property is attractive as less parameters are estimated, largely avoiding overfitting problems. However, without strides, we would apply same parameters to $\{p^{(0)}, v^{(0)}\}$ and $\{p^{(1)}, v^{(1+1)}\}$. In other words, $p^{(i)}$ and $v^{(i)}$ would share same parameters

Because the first layer only captures information at each order block level, we would expect representative features to be extracted when integrating information across multiple order block levels. We can do this by utilizing another convolutional layer with filter size $\{1 \times 2\}$ and stride $\{1 \times 2\}$. The resulting feature maps stem from the micro-view defined by [69]

$$f = \frac{v_k^{(1)}}{v_k^{(1)} + v_k^{(0)}} \quad (7)$$

The weight I is called the imbalance. The micro-price is an important indicator as it considers volumes of bid and ask side, and the imbalance between bid and ask size is a very strong indicator of the next price move. This feature of imbalance has been reported by a variety of researchers [56, 57, 58, 59, 60]. Unlike the micro-price where only the first order book level is considered, we utilize correlations to firm micro-prices for all levels of a LOB so the resulting features maps are of size $[100, 18]$ after two layers with strides. Finally, we integrate all information by using a large filter of size (1×30) and the dimension of our feature maps before the Inception

We apply zero padding to every convolutional layer so the time dimension of our inputs does not change and Leaky Rectifying Linear Units (Leaky-ReLU) [61] are used as activation functions. The hyper-parameter (the small gradient when the unit is not active) of the Leaky-ReLU is set to 0.04, evaluated by grid search on the validation set.

Another important property of correlation is that of equivariance to translation [55]. Specifically, a function $f(x)$ is equivalent to a function g if $f(g(x)) = g(f(x))$. For example, suppose that there exists a main classification feature s located at $(x_{\text{main}}, y_{\text{main}})$ of an image $F(x, y)$. If we shift every pixel of I one unit to the right, we get a new image I' where $F(x, y) = I(x-1, y)$. We can still obtain the main classification feature, s' , of I' and s' is s while the location

classification feature of 18 t' and $18 = 36$, while the location of u' will be $(x_{u'}, y_{u'}) = (x_u - 1, y_u)$. This is important to time-series data, because convolution can find universal features that are decisive to final outputs. In our case, suppose a feature that studies imbalance is obtained at time t . If the same event happens later at time t' in the input, the exact feature can be extracted later at t' .

We do not use any pooling layer except in the Inception Modules. Although pooling layers help to find representations invariant to translations of inputs, the smoothing nature of pooling can cause under-fitting. Common pooling layers are designed for image processing tasks, and they are most powerful when not only can certain features exist in the inputs instead of where they exist [57]. Time-series data has different characteristics from images and the location of representative features is important. Our experiences show that pooling layers in the convolutional layer, at least, cause under-fitting problems to the LSTM data. However, we think pooling is important and new pooling methods should be designed to process time-series data as it is a primitive

- ▶ 输入是 100×40 : 100 次更新、10 档买卖盘价格和数量。
- ▶ CNN: 识别局部买卖盘形状。
- ▶ Inception: 同时看不同时间尺度的局部模式。
- ▶ LSTM: 把局部特征按时间串起来。

DeepLOB 的核心不是“网络很深”，而是给 LOB 的空间结构和时间结构一个合理的模型偏置。

Source: Zhang, Zohren and Roberts.

"Deep Learning for Limit Order Books". Figure 3.

■ 指标 1: MCC 比准确率更稳

- ▶ LOB 标签常常不均衡，很多时候“稳定”类占比高。
- ▶ accuracy 可能被多数类抬高。
- ▶ MCC 可以理解为“预测标签和真实标签的相关性”。
- ▶ 随机预测约为 0，完美预测为 1。

混淆矩阵视角

$MCC = F(\text{全部 true/false positive/negative})$

它不只看“猜对多少”，还看各类之间怎么错。

主论文发现

large-tick 股票在多个 horizon 上更稳定、更容易预测。

主论文关键图：tick 类别改变结果

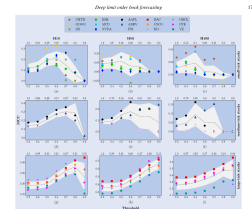


Figure 10. Average Modified Correlation Coefficient (MCC). Results are separated according to the prediction horizons taken into account (one, three, and twelve tick size) and the market conditions (three main pieces of information: (i) the model's performance changes applying different thresholds on the prediction horizons; (ii) the performance changes on the horizon of the model; (iii) the average percentage of winning trades after using the threshold filters on the top of the model; (iv) the performance average scores and the corresponding standard deviations (shown through the grey line and shadows). All the average values and the standard deviations are computed by considering stocks with the same tick size, spanning the 1-year analysis period.

has a value of 0.078. Differently from what happens for the other two classes of stocks, a clear average growing pattern is detected for threshold values ≤ 0.35 , while for average percentage of data used to compute the metric decreases more than what happened at H50, containing, however, higher than the minimum threshold for small- and medium-tick stocks. Overall, large-tick stocks demonstrate greater forecast stability across horizons, as evidenced by the lower variance in MCC. This robustness is strongly linked to their higher liquidity and less brutal spread distributions, which provide a more stable market structure for the DeepLOB model to learn from.

The analysis reported in this Section is further deepened in Appendixes 2 and 3, where we report (i) the one-star MCC of the DeepLOB model at $ROI \in [15, 50, 100]$, for different confidence levels; (ii) the corresponding statistical significance; and (iii) a replica of the analysis in Figure 10 for the FI and accuracy score. The coherence of the results between the robustness of the findings discussed earlier in this Section, highlighting that large-tick stocks exhibit a significant predictability, one across all the considered horizons.

This is evidenced in Figures A1 and A2, where we observe that those stocks where FI score realizations greater than 0.45 without applying any thresholds and surpass 0.7 when probability thresholds are implemented. Similarly, accuracy scores exceed 0.7 without thresholds and reach over 0.9 with the application of probability thresholds. However, as detailed in section 7.2, achieving high scores on these traditional machine learning metrics does not necessarily translate into the generation of successful trading signals. This distinction remarks the complexity of converting predictive accuracy into practical trading strategies.

7.2. On the predictability of model's forecasts

The analysis presented above offers significant insights into the DeepLOB model's performance at different horizons for different classes of stocks. However, further discussion is needed to understand the results from the perspective of the microstructural properties of the LOB. To do so, in this Section, we introduce a novel methodology to evaluate the

- ▶ 行：small / medium / large tick。
- ▶ 列： H_{10}, H_{50}, H_{100} 。
- ▶ 横轴：只保留更高置信度预测。
- ▶ 读图结论：模型表现不是单纯由网络决定，而是和股票微观结构强相关。

Source: Briola et al., Microstructural Guide,

Figure 10.

■ 指标 2：为什么还需要 p_T

- ▶ F1/MCC 是逐样本指标：每个时间点算一次对错。
- ▶ 交易是路径问题：需要开仓、持仓、平仓都大致对。
- ▶ 一个错误如果刚好发生在平仓点，影响比普通点更大。

完整交易概率

$P_T = \{\text{模型预测出的完整交易}\}$

$T_T = \{\text{真实方向对应的完整交易}\}$

$$p_T = \frac{|P_T \cap T_T|}{|P_T \cup T_T|}$$

直觉

p_T 关心“预测序列能否形成正确交易”，而不只是“单点标签是否正确”。

■ 主论文结论：用一句话讲通

1. LOB 可预测性不是模型单独决定的，而是由模型、标签、horizon、tick regime 共同决定。
2. large-tick 股票通常更稳定：价差窄、最优档流动性厚，队列变化更有信息。
3. 传统分类指标有用，但不够；交易信号还要看时间顺序和完整交易路径。
4. 这篇文章真正的贡献是：把深度学习预测重新放回市场微观结构里解释。

适合初学者的记法

先理解市场结构，再评价模型分数；不要反过来只看 leaderboard。

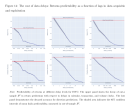
■ 其它论文速览（一）：信号与模型

Ait-Sahalia et al.

- ▶ 高频收益/方向确实存在短寿命可预测性。
- ▶ 延迟会快速消耗预测价值。
- ▶ 作用：解释为什么 LOB 信号必须看极短 horizon。

DeepLOB

- ▶ 经典 CNN + Inception + LSTM 结构。
- ▶ 作用：提供主论文使用的标准模型基线。
- ▶ 局限：不能单独解决跨市场和交易可用性问题。



Source: Ait-Sahalia, Kalnina and Xiu, Figure 14.

■ 其它论文速览（二）：结构与泛化

HLOB

- ▶ DeepLOB 偏向局部卷积结构。
- ▶ HLOB 用图结构描述 LOB 变量间依赖。
- ▶ 作用：说明“市场结构”也可以进入模型设计。

Benchmark Study

- ▶ FI-2010 高分不等于真实市场可迁移。
- ▶ 模型排序会随数据集和 horizon 变化。
- ▶ 作用：提醒我们做跨股票、跨市场验证。



Source: "LOB-Based Deep Learning Models: Benchmark Study", Table 2.

■ 五篇论文合成一个框架

层次	要问的问题	对应论文
信号是否存在	极短 horizon 上是否有可预测性？延迟伤害多大？	Ait-Sahalia et al.
输入怎么表示	LOB 是价格序列，还是买卖盘形状序列？	DeepLOB
市场结构是什么	tick size、spread、depth 是否改变预测难度？	Microstructural Guide
分数是否可信	MCC 高是否代表完整交易可用？	Microstructural Guide
模型能否泛化	换股票、换市场、换 horizon 后排序是否稳定？	HLOB / Benchmark Study

■ 如果自己做 LOB 研究：入门步骤

1. 先定义标签：预测未来几次 LOB 更新后的中间价方向。
2. 再看市场结构： $\bar{\sigma}/\theta$ 、最优档深度、订单簿更新频率。
3. 先跑简单 baseline 和 DeepLOB，不急着上复杂模型。
4. 同时报告 accuracy/F1、MCC、覆盖率，以及类似 p_T 的路径级指标。
5. 最后做跨股票、跨 horizon、跨时间段验证。

原则

模型复杂度应该服务于市场结构解释，而不是替代市场结构分析。

■ 最后的 takeaway

1. LOB 预测的核心对象是“订单簿状态序列”，不是单纯价格序列。
2. tick size 是理解可预测性的入口变量。
3. DeepLOB 是一个好 baseline，但主论文强调的是何时有效、为什么有效。
4. MCC 比准确率更适合不均衡分类； p_T 更贴近交易路径。
5. 真实研究不能只看一个 benchmark 分数。

一句收束

不要先问“哪个模型最强”；先问“这个市场结构下，什么方向信号在什么 horizon 上可预测，并且能否形成完整交易路径”。

■ 讨论问题

- ▶ 如果 MCC 上升但 p_T 下降，这个模型该怎么用？
- ▶ 对 small-tick 股票，应优先改标签、改输入表示，还是改模型结构？
- ▶ 事件时间 horizon 和物理时间 horizon，哪个更适合我们的策略目标？

Q & A

■ 文献与图表来源

- ▶ A. Briola, S. Bartolucci, and T. Aste, “Deep Limit Order Book Forecasting: A Microstructural Guide,” Quantitative Finance, 2025.
- ▶ Z. Zhang, S. Zohren, and S. Roberts, “Deep Learning for Limit Order Books,” IEEE Transactions on Signal Processing, 2020.
- ▶ Y. Ait-Sahalia, I. Kalnina, and D. Xiu, “How and When are High-Frequency Stock Returns Predictable?”
- ▶ A. Briola et al., “Information Persistence and Structure in Limit Order Books.”
- ▶ “LOB-Based Deep Learning Models: Benchmark Study.”

注：本 deck 中的论文图表均来自原论文，仅用于内部研究分享；每页图表下方已标注来源。